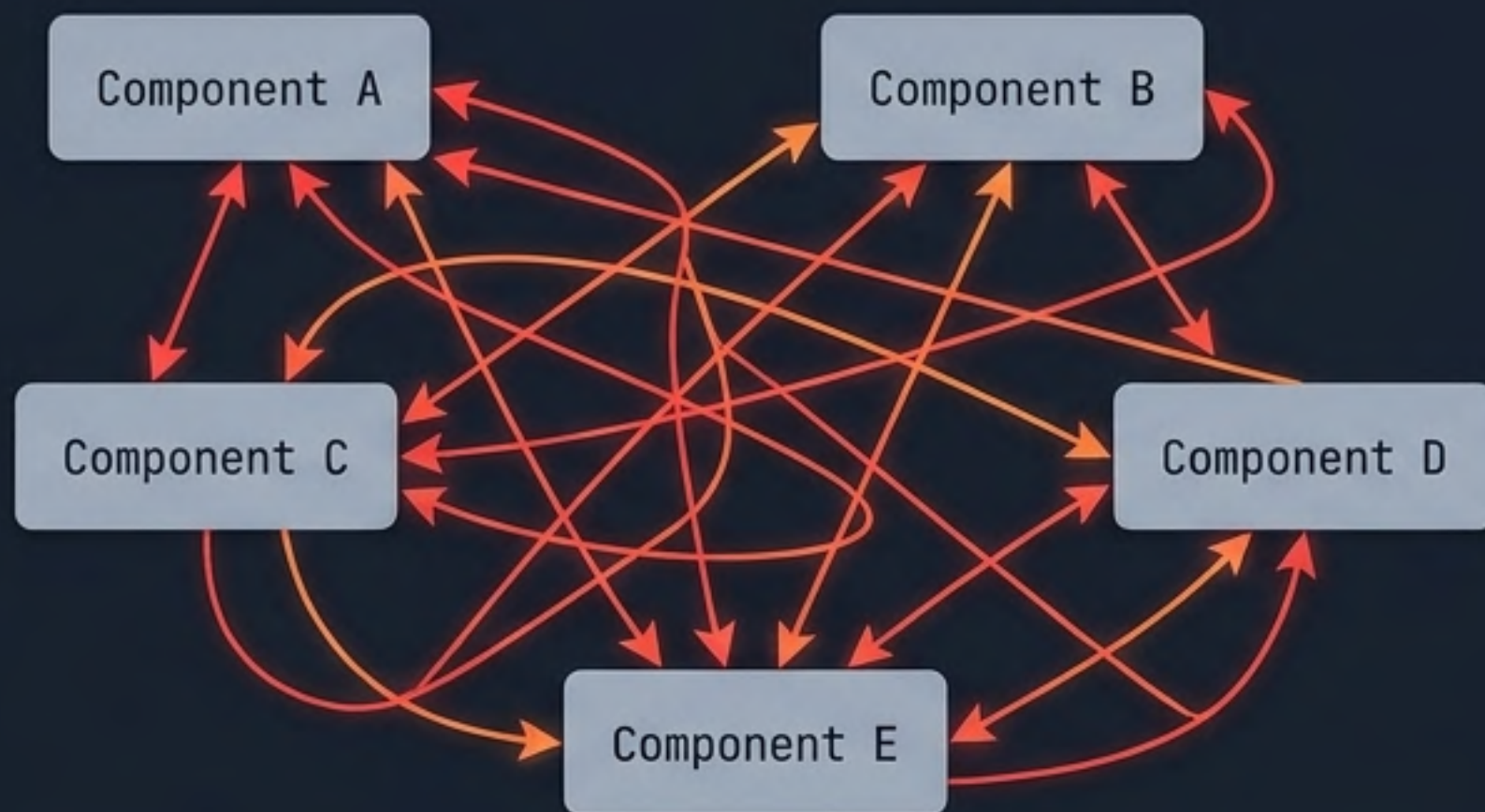


Antes: Spaghetti Data

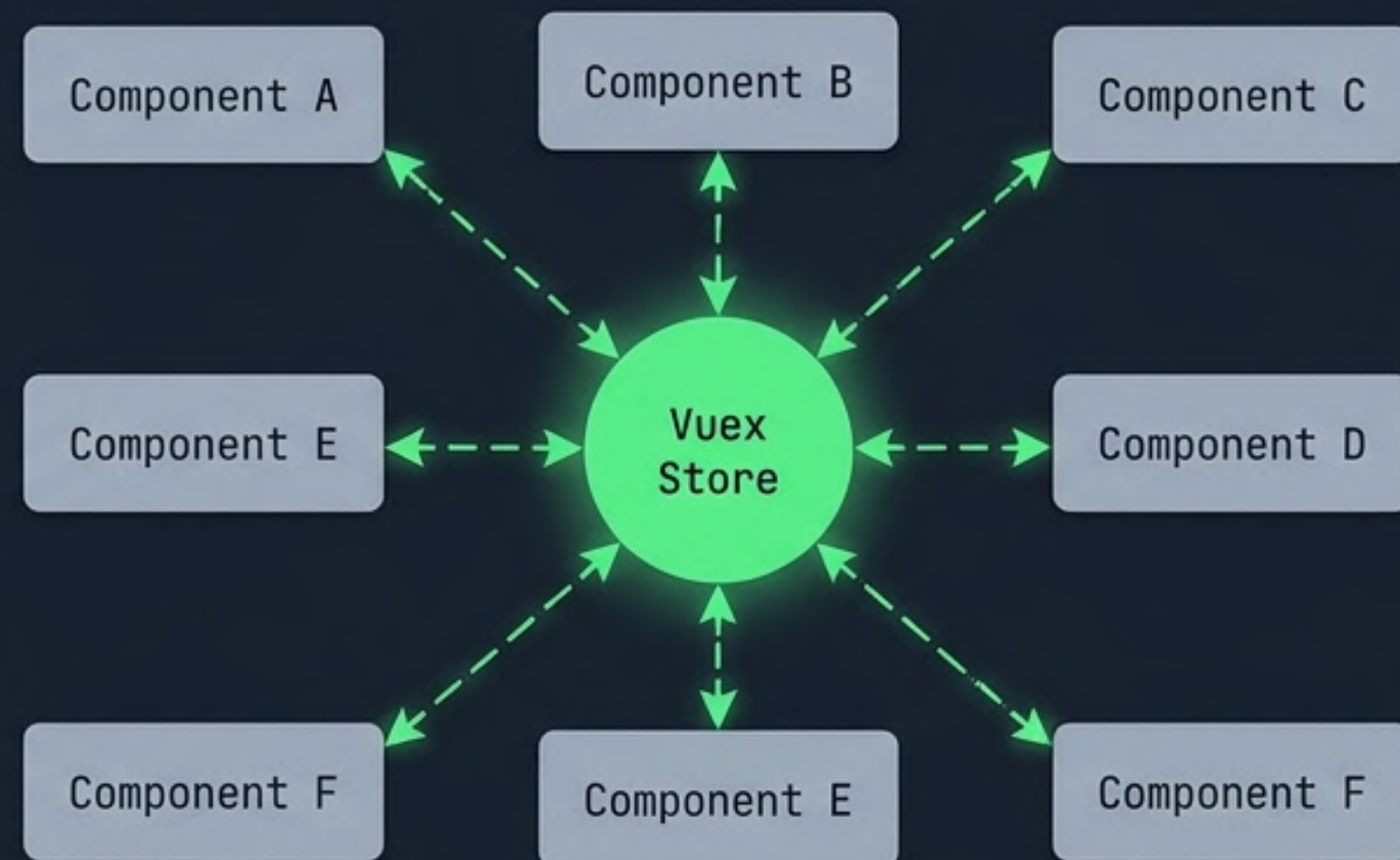


Props Drilling: Pasar datos por múltiples niveles que no los necesitan.

Eventos Caóticos: Emisiones rebotando entre componentes lejanos.

Inconsistencias: Datos duplicados y desincronizados.

Después: El Hub Central



Vuex ordena el tráfico. Extrae el estado compartido de los componentes y lo gestiona en un cerebro global y reactivo.

¿Qué es Vuex?

Definición:

Es un patrón de gestión de estado y una biblioteca clásica de gestión de estado para Vue.js. Actúa como un **Store centralizado** para todos los componentes de una aplicación.

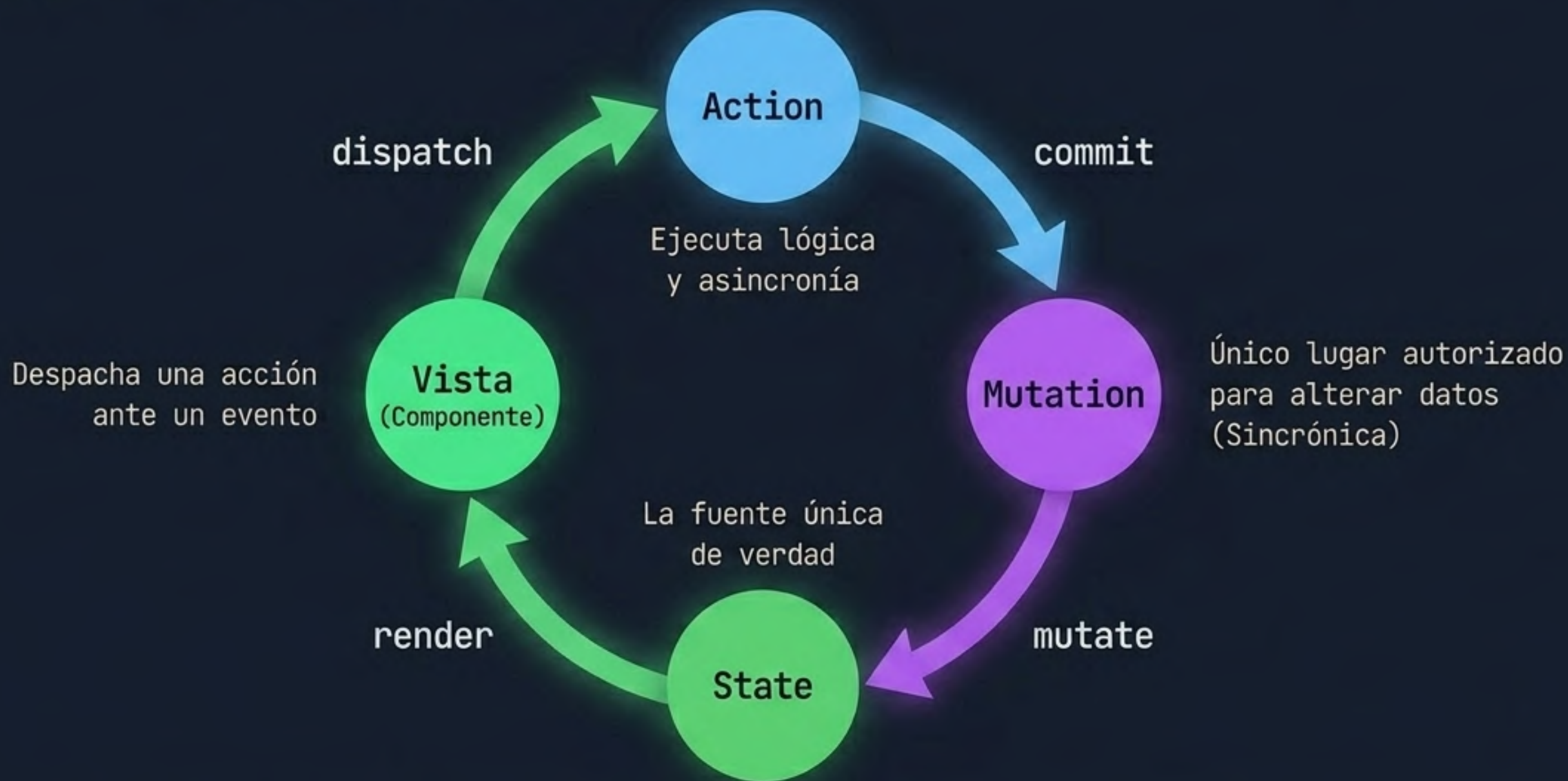
Reglas Estrictas:

Garantiza que el estado solo pueda **mutar** de forma predecible y autorizada.

⚠ ¿Por qué Vuex y no Pinia?

Usamos **Vuex 4** por requerimiento específico del material de estudio. Sin embargo, ten en cuenta que **Pinia** es el estándar moderno recomendado por Vue para proyectos nuevos.

El Flujo Mental de Vuex



El flujo siempre es unidireccional. La vista nunca modifica el estado directamente.

Instalación y Conexión

```
$ pnpm add vuex@4
```

Asegúrate de usar la versión 4, diseñada para Vue 3.

src/main.js

```
import { createApp } from 'vue'
import { createStore } from 'vuex'
import App from './App.vue'

// Creación del store base
const store = createStore({
  state: () => ({}),
  mutations: {}
})

const app = createApp(App)
app.use(store) // Conexión de Vuex a Vue
app.mount('#app')
```

State (La fuente única de verdad)

```
state: {  
  auth: false,  
  username: '',  
  loading: false,  
  error: null  
}
```

Concepto: Un único objeto que contiene todo el estado global de tu aplicación.

Global (Vuex)

Datos compartidos por muchas vistas (ej. sesión, carrito).

Local (ref)

Datos exclusivos de un solo componente: por ejemplo, el texto temporal de un input. No todo debe ir a Vuex.

Getters (Datos derivados)

```
getters: {  
  isAuthenticated: (state)  
    => state.auth  
}
```



🏆 **Regla de Oro:** Son equivalentes a las *computed properties* a nivel global. Leen, pero jamás modifican el `state`. Los resultados se guardan en caché y son reactivos.

Mutations (El único puesto de control)



Concepto: El único lugar en todo el ecosistema de Vuex autorizado para alterar el State.

Regla estricta: Deben ser 100% sincrónicas. Nunca uses `async/await` ni `Axios` aquí.

```
1 mutations: {
2   doLogin(state, username) {
3     state.auth = true
4     state.username = username
5   },
6   doLogout(state) {
7     state.auth = false
8     state.username = ''
9   },
10  setLoading(state, value) {
11    state.loading = value
12  },
13  setError(state, message) {
14    state.error = message
15  }
```


Actions (Lógica y Asincronía)

Aquí vive la lógica de negocio y las llamadas a la API. Las acciones no cambian el estado, despachan (commit) mutaciones.

```
actions: {  
  async doLogin({ commit }, { username, password }) {  
    commit('setLoading', true)  
    commit('setError', null)  
  
    try {  
      if (!username || !password) {  
        throw new Error('Credenciales inválidas')  
      }  
  
      commit('doLogin', username)  
    } catch (error) {  
      commit('setError', error.message)  
    } finally {  
      commit('setLoading', false)  
    }  
  }  
}
```

Permite asincronía (async/await).

Llamadas a APIs externas y lógica de negocio.

Llama a una Mutation autorizada para guardar el éxito o fracaso.

Modules y Namespaced (Divide y Vencerás)

El Problema

Un solo archivo se vuelve inmanejable en aplicaciones reales.

La Solución

Dividir el Store en módulos por dominio de datos (ej. Autenticación por un lado, Datos por otro).

Namespaced (true)

Sin namespace: `dispatch('doLogin')`



Con namespace: `dispatch('auth/doLogin')`



Ensamblando el Store Principal

src/store/index.js

```
import { createStore } from 'vuex'
import auth from '../modules/auth'
import frameworks from '../modules/frameworks'

export default createStore({
  modules: {
    auth,
    frameworks
  },
  strict: import.meta.env.DEV
})
```



Strict Mode Activado

strict: true lanza un error si el estado se muta fuera de una Mutation (ej. modificando el estado directamente desde una Vista). Es tu mejor herramienta para aprender y hacer cumplir las reglas, solo activo en desarrollo.

Axios Centralizado

src/api/api.js

```
import axios from 'axios'
```

```
export const api = axios.create({  
  baseURL: 'http://localhost:3000',  
  timeout: 5000  
})
```



Regla de Oro: Las vistas **NO** llaman a Axios directamente cuando esos datos deben vivir en el estado global. La vista despacha una action y la action usa Axios.

Módulo 1: Auth (auth.js)

```
export default {
  namespace: true,
  state: () => ({
    auth: false,
    username: '',
    loading: false,
    error: null
  }),
  getters: {
    isAuthenticated: (state) => state.auth
  },
  mutations: {
    doLogin(state, username) {
      state.auth = true
      state.username = username
    },
    doLogout(state) {
      state.auth = false
      state.username = ''
    },
    setLoading(state, value) {
      state.loading = value
    },
  },
```

```
    setError(state, message) {
      state.error = message
    }
  },
  actions: {
    async doLogin({ commit }, { username, password }) {
      commit('setLoading', true)
      commit('setError', null)
      try {
        if (!username || !password) {
          throw new Error('Credenciales inválidas')
        }
        commit('doLogin', username)
      } catch (error) {
        commit('setError', error.message)
      } finally {
        commit('setLoading', false)
      }
    },
    doLogout({ commit }) {
      commit('doLogout')
    }
  }
}
```


Módulo 2: Frameworks (frameworks.js)

```
import { api } from '@api/api'

export default {
  namespaced: true,
  state: () => ({ items: [], loading: false, error: null }),
  mutations: {
    setItems(state, items) { state.items = items },
    setLoading(state, value) { state.loading = value },
    setError(state, message) { state.error = message }
  },
  actions: {
    async cargar({ commit }) {
      commit('setLoading', true)
      commit('setError', null)
      try {
        const { data } = await api.get('/frameworks')
        commit('setItems', data)
        commit('setItems', data)
      } catch (error) {
        commit('setError', error.message)
      } finally {
        commit('setLoading', false)
      }
    }
  }
}
```


Vue Router y Rutas Protegidas



Usamos el store dentro de los Navigation Guards del router para bloquear el acceso a usuarios sin sesión. Modulariza cuando el store crezca o cuando tengas dominios claros como auth, cart, frameworks.

router/index.js

```
import store from '@store'

// ... configuración de rutas con meta: { requiresAuth: true }

router.beforeEach((to) => {
  const isAuthenticated = store.getters['auth/isAuthenticated']

  if (to.meta.requiresAuth && !isAuthenticated) {
    return '/login' // Bloqueado, ir a login (Vue Router 4+)
  }
})
```


Flujo de la App y Reglas de Oro

Flujo de la Mini-App (Usando <script setup>)

LoginView.vue

Formulario usa ref local. Al enviar
-> `store.dispatch('auth/doLogin')`
-> `router.push('/dashboard')`

DashboardView.vue

Al montar
-> `store.dispatch('frameworks/cargar')`

La vista lee datos reactivos usando:
`computed(() => store.state.frameworks.items)`

Las 4 Reglas de Oro



Nunca mutar el **state** desde la vista. Desde la vista, prefiere **dispatch** para ejecutar **actions**. Las **actions** harán **commit** cuando necesiten cambiar el **state**.



Cero asincronía en **Mutations**. Todo **async/await** pertenece a las **Actions**.



No **Axios** directo en vistas. Si el dato va al **Store** global, el **Store** hace la llamada.



Modulariza siempre. Usa **modules** y **namespaced: true** desde el primer día.